Outlook Calendar Intelligence

End-to-end workflow, the enrichment agent's logic, and how pre-meeting notes are produced

Repo: `outlook-calendar-poc` · **Branch:** `feat-outside-bis-physician-pre-notes` (level with `main`) · **Generated:** 2026-09-01 · **Source:** `docs/system-workflow.html`

1. What this system actually is

The repo is named "Outlook Calendar POC", but it is a **pre-meeting sales-intelligence platform for a medical-device rep** (Lumendi context). The rep never leaves Outlook. When a meeting with a physician appears on the calendar, the system identifies who the meeting is with, assembles everything it knows about them, and puts that brief in front of the rep in three places — as an email, inside the meeting itself, and in the web app.

Surface	What it does	Code
Web app	Day's calendar, expandable meeting cards, in-app brief, notes, Email Intelligence Sheet, Dynamics leads	`public/{index.html,app.js}` · `src/routes/api.routes.js`
Email	Instant brief when a meeting appears; reminder brief before it starts; external (non-BIS) brief	`src/graph.js` · `src/reminders.js`
The meeting itself	Brief injected into the event body — visible when the rep opens the meeting on any device, no add-in needed	`graph.injectBriefIntoEvent`
Outlook add-in	Ribbon task pane reading attendees via Office.js (blocked on personal outlook.com accounts)	`src/routes/addin.routes.js` · `public/addin`
Dynamics 365	Lead → BIS physician brief in a side pane (token-gated iframe)	`src/routes/embed.routes.js` · `src/dynamics.js`

2. The runtime — one web process, two background engines

`server.js` boots Express (session in SQLite, or Redis when `REDIS_URL` is set), waits for the physician directory to load from Supabase, mounts the routes, and then — only when run as a long-lived process — starts two timers. Everything the rep receives without clicking anything comes from these two loops.

Engine	Interval	Job
`src/email-ingest.js` "ingest tick"	`INGEST_POLL_SECONDS` default **300 s**	Sync calendar → `app_activities`; instant-brief new physician meetings; inject briefs into meeting bodies; run the enrichment agent on meetings that match nobody; ingest inbox + sent replies; AI meeting notes; heal the Email Intelligence Sheet
`src/reminders.js` "reminder tick"	`REMINDER_POLL_SECONDS` set to **60 s**	For every signed-in rep, find meetings starting within `REMINDER_LEAD_MINUTES` (set to **30**) and email one reminder + full brief per meeting

Both loops iterate **every signed-in salesperson** from the persisted token store (`src/token-store.js`) and silently refresh each one's Microsoft access token (`auth.getAccessTokenForUser`), so they keep working with nobody logged into the browser. Both are individually killable (`INGEST_ENABLED=false`, `REMINDERS_ENABLED=false`), and the ingest engine no-ops entirely without Supabase.

> **Why polling, not webhooks.** Graph change notifications need a public HTTPS endpoint; the handler exists (`/auth/webhooks/outlook`) but on localhost the same logic runs on a poll so the whole pipeline is testable in dev. Every side effect is guarded by an idempotency key (§6.4), so poll frequency changes latency, never duplicates.

3. The master workflow

Microsoft Graph — the rep's calendar
graph.getUpcomingEvents(token, 30 days) · calendarView, UTC-normalised, one event per recurring occurrence

▼

Who is this meeting with?
context.attendeesToEnrich(event) — attendees only, never the organizer, never resource/room mailboxes → physicians.getByEmail(exact match in the 21k BIS directory)

▼

MATCHED in BIS
one or more attendee emails are in `bis_physicians`

▼

Link + brief
upsert `app_activities` (first NPI) → assemble bundle per physician → **instant brief email** (first sight only) → **inject brief into the meeting body**

NO match in BIS
the old dead end: no activity, no brief, rep walked in blind

▼

Enrichment agent (§5)
briefUnknownAttendees → subjects = attendees, or the people *named in the title* when there are none → `enrichment.enrich()` cascade

▼

Agent outcome decides what the rep gets
`recovered_in_bis` → link the activity and brief them exactly like a matched physician (email + meeting body) · `external` → provenance-tagged brief by email *and* injected into the meeting · `ambiguous` / `unresolved` / `not_physician` / `lookup_failed` → nothing mailed, logged only

▼

Reminder tick, independently
REMINDER_LEAD_MINUTES before the meeting: one email, all physicians on that meeting, same brief body, one-per-meeting sent-log

SAME INGEST TICK — THE MAIL SIDE OF THE LOOP

Inbox delta
graph.getInboxDelta(deltaLink) — only what's new since last tick

Sent Items
getRecentSent(25), kept only if the subject is a reply (`RE:`)

▼

cleanBody()
cut at the first quote marker ("On … wrote:", "From:", "_____"), drop ">" lines, trim the signature block → just what the sender wrote

▼

app_emails
immutable row, deduped on (provider, provider_msg_id), linked

AI MOM → Meeting Notes
ai-extractor reads the reply → note saved with `source:'ai'` against (NPI, rep)

Email Intelligence Sheet
intel-extractor → physician/facility/CPT row + "new

to an activity by thread → sender physician → subject

vs `bis_*` ” flags; reconcile pass heals missing rows

4. Stage-by-stage: what one ingest tick does

`syncActivities(token, user)`, per event in the 30-day window (`ACTIVITY_WINDOW_MAX`), all-day events skipped:

1. **Resolve physicians** — `physiciansForEvent()`: every attendee whose email is an *exact* hit in the directory. Two physician attendees → two matches, both briefed in one email.
2. **Was it already known?** `crm.findActivityByEventId()` is checked *before* the upsert — this is what distinguishes “a meeting just appeared in Outlook” from “the same meeting on the 40th poll”.
3. **Upsert the activity** — `app_activities` holds one NPI per row, so the first match is the link; all matches are still briefed.
4. **Instant brief** (new meetings only) — `sendInstantBrief()`, keyed on the *series*, so one weekly recurring meeting sends one brief and not 29.
5. **No match → the agent** — `briefUnknownAttendees()`. Deliberately *not* gated on “new”, so meetings that predate the feature are covered; its own `enrich:` key keeps it to once per meeting/series.
6. **Inject the brief into the event body** — `enrichEventBody()` for every physician meeting, new or old; the key plus the in-body marker keep it to exactly once.

Then `ingestEmails()` (inbox delta), `ingestSentReplies()` (the rep’s own “RE:” mail, which the inbox delta never sees), and `reconcileIntel()` — a self-heal pass that re-scans the last `INTEL_RECONCILE_DAYS` (3) days of inbox and re-processes any message that still has no sheet row, because the delta cursor will never resurface it.

> **The identity rule the whole pipeline rests on.** A brief is only *mailed or injected* off an **exact attendee email match**. The organizer is never matched. The title and description are used for *context* (facility, city, state) and, when a meeting has no attendees at all, as a *candidate name* that must survive the full agent cascade — never as an identity on their own. A wrong guess here emails one physician’s commercial data under another physician’s name.

5. The agent — how it works and why

`src/enrichment/index.js` is the agent. One entry point, `enrich(query)`, wrapped in a per-call health ledger. It exists to answer one question the master data cannot: *who is this person, and what should the rep know about them, when they are not in `bis_physicians`?*

5.1 Design principles it enforces in code

- **BIS wins, always.** Nothing here writes to `bis_*`; the master stays read-only. External data decorates, never overwrites.
- **Free registries first, AI last.** The paid tier runs only for the one job registries cannot do — turning an opaque email address into a name.
- **Every claim is attributable.** Fields are `{value, source, sourceUrl, tier, confidence, retrievedAt}` end to end (`provenance.js`); a web field with no proof URL is not rendered.
- **A weak guess is worse than nothing.** Below the accept threshold the answer is downgraded to a candidate list, and the fields that rested on that identity are *dropped*.
- **Degrade, never crash.** Every source is optional; the agent distinguishes “the registry said no” from “the registry never answered”.

5.2 The cascade

T0 — **BIS directory by email.** A hit means `status:in_bis`, confidence 100 — return immediately, the standard brief applies. This is the fast exit for the 21k physicians already in the master. *free · 0 ms*

CACHE — **Recent-lookup cache** (`enrichment/cache.js`), checked *after* BIS so the master is always read live. Never serves an answer shallower than the request wants (a free-tier result can't satisfy a paid-tier request). `unresolved` and `degraded` results are never cached. *free*

T0.5 — **Email-domain index.** `@med.unc.edu` → the facility that BIS's own physicians at that domain share, with the count as evidence. Gives a facility, city and state before anything external is touched. *free · 0 ms*

T1 — **Web identity (Claude + web search).** The only paid tier, and the only thing that can turn `nshaheen@med.unc.edu` into "Nicholas Shaheen, MD, MPH, Gastroenterology, UNC". Runs only when there is something to buy: no caller-supplied name, the mailbox isn't organisational/shared, `ANTHROPIC_API_KEY` is set, and `useWeb ≠ never`. *paid · ~$0.15 · ~14 s*

T2 — **NPPES.** Name + state → NPI, specialty, credential, phone, practice address, taxonomies, licence. Up to 3 interpretations of the address ("nshaheen" → N. Shaheen / Shaheen N. / surname Nshaheen) are searched *in parallel* — sequentially this cost ~17 s on an address that resolved to nothing. *free*

T3 — **Re-match into BIS by NPI.** The highest-value step: the physician *is* in the master, their email simply wasn't. `status:recovered_in_bis`, confidence ≥ 90 — and the meeting is then linked and briefed through the completely normal path. *free*

T3b — **Facility re-match.** When the person can't be recovered, match their CMS hospital (or the title's facility hint) into `bis_facilities` — city/state gated — which yields territory, health system, and **colleagues already in BIS at that facility**: the most useful thing to hand a rep when the person themselves is unknown. *free*

T4 — **Decoration, four sources at once:** CMS hospital affiliation (type, ownership, star rating, CCN), **CMS Open Payments** (who else is already paying this physician — BIS has no column for this), PubMed publications, ClinicalTrials.gov studies. *free · parallel*

5.3 How confidence is computed

`pickBestProvider()` scores every NPPES hit against everything else the agent knows, then `runEnrich()` discounts by how trustworthy the *name* that produced the hit was.

Signal	Δ score	Note
A registry hit at all	**base 30**	starting point
First name exact / prefix / differs	+40 / +20 / −25	
Only surname hit whose first initial matches (initial-only hint)	+25	uniqueness is real evidence

First initial matches, but so do others / doesn't match	+12 / –10	
State match / city match	+25 / +20	geography is what separates same-name providers
NPI status active · only registry hit	+5 · +15	
Runner-up within 15 points → **ambiguous**	capped at 55	"we cannot tell these two apart" is said, not hidden
Name-source discount	× (0.6 + 0.4·c/100)	a guessed name is worth less than a rep-supplied one — blended, not multiplied, so a strong unique hit still corroborates the guess

Two thresholds then decide what the rep is told: `CONFIDENCE_ACCEPT = 70` (present as resolved) and `CONFIDENCE_SUGGEST = 40` (present as "possible match — confirm"). Below 40 the identity claim is **withdrawn**: the NPI is dropped, every field that came from that identity is stripped from the profile, and only candidates remain. A facility that stands on its own evidence (the email domain) survives; one that came from the discarded NPI does not.

5.4 The statuses — the agent's whole vocabulary

Status	Meaning	What the rep gets
`in_bis`	Already in the master (T0)	**standard brief**
`recovered_in_bis`	In the master under a different email; found by NPI (T3)	**standard brief + injected**
`external`	Genuinely outside BIS, resolved ≥ 70	**provenance-tagged brief + injected**
`ambiguous`	Best match 40–69	**in-app suggestion only — never mailed**
`facility_only`	Person unresolved, facility identified	**facility + BIS colleagues**
`not_physician`	Web tier is ≥ 50 % sure they're an admin/rep/researcher	**nothing — cached so the same colleague isn't re-bought**
`unresolved`	Sources answered; none identified anyone	**nothing**
`lookup_failed`	An *identity* source never answered (DNS/outage)	**"could not reach the registry", explicitly not "not found"**

> **The distinction that took real work.** A resolver that cannot reach `npiregistry.cms.hhs.gov` used to produce a confident "could not resolve this address from the free registries" — a claim about the physician made on evidence about the network. The health ledger (`enrichment/health.js`) is read *before* the status is decided: blind identity sources ⇒ `lookup_failed` , degraded runs are never cached, and every profile carries a note naming the source that was missing. `npm run enrich:doctor` diagnoses it.

5.5 The other AI in the system

All of it is Claude `claude-opus-4-8` via `@anthropic-ai/sdk` , and all of it no-ops gracefully when `ANTHROPIC_API_KEY` is unset.

Module	Pattern	Job

`enrichment/sources/web-identity.js`	2 calls: `web_search` with `tool_choice:auto`, then forced tool-use over the findings	email → identity + **proof URLs**. Uses the basic `web_search_20250305` on purpose: the newer variant returned zero citations, and the citations *are* the feature.
`ai-extractor.js`	forced tool-use, strict JSON	one email reply → MOM points saved as an AI meeting note
`intel-extractor.js`	forced tool-use, strict JSON	email → CPTs, facility, key info for the Email Intelligence Sheet
`entity-matcher.js`	**no AI** — rule/strategy chain	free text → people, facilities, locations, matched to master data (exact → alias → initials → token → fuzzy), threshold 75, else top-5 suggestions
`scripts/ingest-products.js`	forced tool-use	Lumendi brochure PDFs → `config/product-context.json`

> **Known model gotcha, handled.** The model can leak tool-call delimiter fragments into string values (especially around empty fields). Both `intel-extractor.js` and `web-identity.js` run a `sanitize()` / `cleanField()` pass that truncates at the first delimiter fragment. Keep it.

6. Pre-meeting notes — the actual deliverable

6.1 What a brief contains

One function builds it — `graph.physicianBriefHtml()` — and the email, the in-app brief, the injected meeting body and the embeds all call it, so they can never drift apart.

Section	Content	Source
Physician details	Name, NPI, specialty, facility, phone, email **with a trust badge**	`bis_physicians + app_contacts`
Data check	BIS vs NPPES by NPI — flags a physician who has moved, so a stale facility/email is never presented as fact	`enrichment/verify.js`
Contact intelligence	Confirmed contact details, role, relationship notes	`app_contacts`
Procedure intelligence	CPT volumes grouped into procedure families	`bis_procedure_volumes`
Commercial signals	Reimbursement-weighted signals + this physician's Lumendi account status	`bis_cpt_reimbursement · app_accounts`
Account opportunity	Facility-level opportunity vs peer physicians	`analytics.getAccountOpportunity`
Product fit	Best-fit DiLumen product, acuity-tiered (ESD → C1, EUS → EUS accessory, EMR → EZ1, colon → EZ Glide)	`product-fit.js`
Talking points	Product context matched to the physician's procedure families	`config/product-context.json`
Meeting notes	This rep's note history with this physician — human notes and AI-extracted MOM	`notes.js (SQLite/Redis)`

For someone **outside** BIS, `graph.externalBriefHtml()` renders the agent's profile instead: the same reading order, but every row carries its source label and a clickable proof URL, plus the extras BIS does not model at all — industry payments and who is paying, publications, trials, facility star rating, licence, and the agent's own reasoning.

6.2 The three ways notes reach the rep

Channel	When	Detail
Instant brief email `sendInstantBrief`	First time a physician meeting is seen	Covers meetings created *directly in Outlook*, which never touch the app's schedule route. All physicians on the meeting in one email, a section each.
Injected into the meeting `injectBriefIntoEvent`	Same tick, once per event	`PATCH /me/events/{id}` **prepends** the brief to the body inside a marker block. Updates only the organiser's copy — attendees are never notified — and preserves the existing body (Teams join info, etc.). Works on any account, including personal outlook.com where add-ins are blocked.

Reminder email `reminders.tick`	`REMINDER_LEAD_MINUTES` before the start	Independent engine and independent dedupe, so the reminder fires whether or not the instant brief did. Adds the physician the meeting was *scheduled* with (from `app_activities`) to the attendee matches. Meeting time is UTC-stamped by `formatMeetingTime()` so it reads identically to the other briefs.

6.3 Meeting notes, human and AI

`src/notes.js` keeps one note per row against **(physician NPI, rep email)** — every rep keeps their own history with each physician — in SQLite by default, Redis when `REDIS_URL` is set. Notes carry `source: 'human' | 'ai'`. The AI ones come from the ingest loop: a reply is cleaned, the physician is resolved (linked activity → sender → entity match on the subject), the extractor turns it into MOM points, and the note is written with the meeting's `eventId` so the UI can scope it to that meeting. Only replies (and mail a physician sent directly) are summarised, so the app never summarises its own outgoing brief.

6.4 Idempotency — why nothing arrives twice

Key / guard	Protects
`instant:<seriesKey(ev, physicians)>`	The instant brief. Keyed on Graph's `seriesMasterId` + a fingerprint of the people, so one weekly meeting sends **one** brief instead of ~29 — while an occurrence edited to be with a different doctor still gets its own.
`enrich:<seriesKey(ev, subjects)>`	The agent run — including the paid tier. Same series logic; this is the key that stops a recurring meeting buying ~29 identical lookups.
`enriched:<eventId>`	The meeting-body injection (skips the Graph round-trip on later ticks).
In-body `BRIEF_MARKER`	Second line of defence for the injection — the body itself is checked before writing.
Reminder sent-log per `(user, eventId)`	Exactly one reminder per meeting per rep, persisted so a restart doesn't re-send.
`(provider, provider_msg_id)`	Email ingestion and the AI note — extraction runs only on a genuinely fresh insert.
Mail `deltaLink` per user	The inbox is never re-read; `reconcileIntel()` exists precisely because that cursor cannot go backwards.

7. Data, config and current state

7.1 Data model (Supabase, `src/supabase.js`)

Table / store	Role
`bis_physicians (~21k)`, `bis_facilities (~12.8k)`, `bis_procedure_volumes (~321k)`, `bis_cpt_reimbursement`	**Master data, read-only.** Plus RPCs `bis_directory`, `bis_physician_analytics`, `bis_search_physicians`.
`app_activities`	Meetings, one physician NPI each, keyed to `calendar_event_id`
`app_emails`	Ingested mail, immutable, linked to an activity when resolvable
`app_contacts / app_accounts`	Confirmed contact details and Lumendi account status (POC-curated, ~12 rows each)
`app_email_intel`	The Email Intelligence Sheet — **exists in DEV only**
`app_audit_log`	Every system/AI action (`email.ingested`, `insight.extracted`, …)
`data/notes.db · sessions.db`	Meeting notes and sessions (SQLite, or Redis when configured)

`SUPABASE_ENV=development|production` flips the entire app between the dev and prod Supabase projects; the app uses the anon key, and DDL is run by hand from `supabase/*.sql`.

7.2 The knobs that change behaviour

Variable	Now	Effect
`REMINDER_LEAD_MINUTES`	30	How early the reminder brief lands — the single timing knob
`REMINDER_POLL_SECONDS`	60	Scan frequency; a meeting inside the lead window is briefed on the next poll
`INGEST_POLL_SECONDS`	300 (default)	How fast a new Outlook meeting gets its instant brief and injection
`INTEL_RECONCILE_DAYS`	3	Self-heal window for the Email Intelligence Sheet
`EMAIL_INTEL_DAYS`	10	History the backfill reads
`BRIEFING_TO_EMAIL`	unset	Unset on purpose, so each rep gets their own brief in their own mailbox
`ANTHROPIC_API_KEY`	set	Unset ⇒ every AI feature (incl. the paid T1 tier) no-ops; free tiers still work
`INGEST_ENABLED / REMINDERS_ENABLED`	on	`=false` kills either engine independently

7.3 Where the work currently stands

The branch `feat-outside-bis-physician-pre-notes` is level with `main` and the tree is clean; the last stretch of commits were correctness fixes on top of the agent rather than new surface area:

- recurring series deduped once instead of once per occurrence (agent *and* instant brief);
- an unreachable source told apart from an empty one (`lookup_failed`);

- “a run of capitalised words is a project, not a person” — title-name false positives;
- brief now says how far the contact details can be trusted, and a meeting that only *names* someone gets briefed;
- UI: physician search falls back to enrichment, directory load retries, collapsed meeting cards name who they can look up.

Open items

- `app_email_intel` is DEV-only — run `supabase/email-intel-setup.sql` (plus the contacts/accounts setup SQL) in PROD before going live.
- NPPES DNS (`npiregistry.cms.hhs.gov`) has been dropped by the local router before; that is what `lookup_failed` and `npm run enrich:doctor` are for.
- Product-fit mapping is hardcoded in `product-fit.js` (scorer reads a catalog constant, not the DB); `app_contacts` / `app_accounts` are curated POC rows, not a CRM integration.
- POC RLS is anon-all; the Dynamics client secret lives only in `.env` and should be rotated before prod.
- Add-in cannot be sideloaded on a personal outlook.com account — the meeting-body injection is the answer that works everywhere.
- Verification is `node --test test/*.test.js` (5 targeted suites), `node --check`, and live runs — there is no broad test suite.

7.4 File map

Path	Responsibility
`server.js`	Express boot, session, route mount, starts both engines
`src/auth.js · token-store.js`	MSAL OAuth; persisted tokens + dedupe/sent logs so engines run headless
`src/graph.js`	All Graph I/O **and** every brief renderer (email = in-app = injected)
`src/email-ingest.js`	The ingest tick: activities, instant briefs, injection, mail, AI notes, intel heal
`src/reminders.js`	The reminder tick
`src/enrichment/*`	The agent: `index.js` (cascade), `context.js` (who/what a meeting says), `rematch.js`, `verify.js`, `provenance.js`, `cache.js`, `health.js`, `sources/*`
`src/analytics.js · product-fit.js · product-context.js · territory.js`	The commercial half of the brief
`src/entity-matcher.js · physicians.js`	Text → entities → master-data matches; the 21k directory
`src/email-intel*.js · intel-extractor.js`	Email Intelligence Sheet
`src/routes/*`	`api` (app), `auth` (+ webhook), `embed` (Dynamics, token-gated), `addin` (Outlook task pane)